

## 7일 차

# 클래스 선언 방식

### 이번 차시에서 주목할 키워드

- 클래스
- `__init__` 메서드
- `forward` 메서드

## 7일 차 학습 목표

- 클래스의 기본 개념을 이해하고 객체지향 방식의 딥러닝 모델을 구현할 수 있습니다.
- 기본형, 중첩형, 모듈화형 등 다양한 클래스 선언 방식을 살펴봅니다.
- `__init__`와 forward 메서드의 역할을 이해하고 모델 클래스를 직접 설계할 수 있습니다.

이번 차시에서는 클래스(Class)의 개념을 이해하고 지금까지 배운 회귀와 분류 모델을 클래스로 구현합니다. 또한 다중 입력 모델을 클래스로 구현합니다. 모델을 클래스로 만들면 코드의 재사용성과 확장성이 높아지고 유지보수가 한결 쉬워집니다.

## 클래스 이해하기

딥러닝 모델링을 위해 클래스(Class)의 기본 개념과 필요성을 쉽고 간결하게 살펴봅니다. 클래스의 복잡한 이론보다는 딥러닝 모델을 효율적으로 관리하고 확장하기 위해 꼭 필요한 내용 중심으로 소개합니다.

딥러닝 모델을 만들다 보면 코드가 점점 길어지고 복잡해지는 경우가 많습니다. 초기에는 `nn.Sequential` 방식이 모델을 정의하고 데이터를 학습시키는 데 간편하고 직관적으로 느껴질 수 있습니다. 하지만 모델에서 기능이 추가되고 학습에서 하이퍼파라미터를 조정하는 등의 작업이 진행되면 점차 코드의 복잡성이 빠르게 증가해 코드를 관리하기가 어려워집니다. 이때 클래스를 사용하면 코드의 재사용성이 높아져 유지보수를 훨씬 더 쉽게 수행할 수 있습니다.

이번 차시에서는 딥러닝 모델링에서 클래스를 왜 사용해야 하는지, 그리고 클래스로 코드의 효율성과 확장성을 어떻게 높이는지 알아봅니다.

## 클래스의 필요성

### 함수 기반 코드의 한계

딥러닝 모델을 처음 구현할 때는 보통 데이터를 불러오고 모델을 정의하고 학습과 평가를 수행하는 과정을 각각 함수로 작성합니다. 이 방식이 처음에는 직관적이고 간단해 보이지만, 모델을 개선하거나 기능을 확장하는 과정에서 다음과 같은 한계가 드러납니다.

- 코드의 중복 증가: 모델 구조를 조금만 바꿔도 함수 전체를 다시 작성해야 하는 경우가 많습니다. 예를 들어 은닉층을 하나 추가하거나 활성화 함수를 바꾸려면 모델 정의 함수 전체를 복사해 새로운 버전을 만들어야 합니다. 실험을 반복할 수록 코드가 여러 버전으로 흩어져 관리하기가 어려워집니다.
- 유지보수의 어려움: 학습률, 드롭아웃 비율, 배치 크기 등 하이퍼파라미터를 바꾸려면 코드를 일일이 찾아 수정해야 합니다. 함수가 분리되어 있지만 서로 의존성이 있는 경우, 일부만 수정하고 빠뜨리는 실수를 하면 학습이 원하는 대로 진행되지 않을 수 있습니다.
- 확장과 재사용의 제약: 새로운 모델 구조를 추가하거나 일부 기능만 바꾸고 싶어도 기존 함수 코드를 대폭 수정해야 합니다. 예를 들어 같은 네트워크에서 층(Layer)만 다르게 적용하거나 특정 단계에서만 연산을 추가하고 싶어도 함수 기반 코드로는 유연하게 대응하기가 어렵습니다.

함수 기반 코드는 간단한 실험에서는 편리하지만, 실제 프로젝트처럼 모델을 계속 발전시키고 다양한 실험을 반복해야 하는 상황에서는 관리하기가 까다롭습니다. 따라서 함수 기반 코드의 한계를 보완하고 코드의 재사용성과 확장성을 높이기 위해 클래스를 많이 사용합니다.

### 클래스 사용의 장점

주로 모델 구조에 한정해 클래스를 사용하는 이유는 다음과 같습니다.

- 코드의 중복 감소: 모델 구조가 비슷하고 일부만 바뀌는 경우 클래스 내부에서 파라미터를 변경하는 방식으로 쉽게 새로운 모델을 정의할 수 있습니다. 클래스를 이용하면 불필요한 코드 복사를 줄일 수 있습니다.
- 유지보수의 용이성: 모델 구조나 하이퍼파라미터가 바뀌더라도 클래스 내부의 메서드만 수정하면 됩니다. 학습 루프나 평가 코드를 따로 건드리지 않으므로 실수를 줄이고 일관성을 유지할 수 있습니다.
- 확장성 확보: 기존 클래스를 상속받아 새로운 기능을 추가하거나 일부 메서드만 재정의(Override)할 수 있습니다. 예를 들어 기본 모델 클래스가 있다면 드롭아웃이 추가된 버전을 새로 만들 때 새로운 모델을 다시 만드는 게 아니라 기존 코드를 그대로 활용할 수 있습니다.

- 협업과 가독성 향상: 모델 구조와 동작을 하나의 클래스에 묶으면 코드의 흐름이 명확해집니다. 다른 팀원이 모델을 수정하거나 기능을 추가할 때도 필요한 부분을 한눈에 파악할 수 있어 협업이 훨씬 수월합니다.
- 실험 자동화에 유리: 파라미터만 바꿔 여러 모델을 순차적으로 학습시키는 스크립트를 작성할 때 클래스 기반의 모델 정의는 특히 강력합니다. 동일한 학습 코드로 다양한 모델을 손쉽게 불러와 실험할 수 있습니다.

결국 클래스는 딥러닝 프로젝트에서 필수적인 '틀' 역할을 합니다. 한 번 잘 만들어두면 모델을 바꿀 때마다 코드를 다시 짜는 대신, 클래스 초기화 메서드(객체를 생성할 때 자동으로 실행되는 준비 작업)나 메서드의 인자(메서드가 받을 입력값)만 바꿔 다양한 실험을 수행할 수 있습니다. 이는 점점 복잡해지는 모델링 과정에서 시간을 절약하고 코드의 품질을 높이는 가장 효과적인 방법입니다.

## 클래스의 기본 개념

클래스는 객체 지향 프로그래밍의 핵심 개념으로서 데이터(속성)와 기능(메서드)을 하나로 묶어 코드를 효율적으로 관리하도록 도와주는 도구입니다. 딥러닝 모델을 클래스 기반으로 작성하기 전에 먼저 클래스의 기본 개념을 이해할 수 있는 쉬운 예제를 하나 소개합니다.

### 예제: 강아지 클래스

강아지(Dog)는 이름, 나이, 종(품종)과 같은 속성(특징)이 있고, 짖는다(bark), 먹는다(eat)와 같은 동작(기능)을 합니다. 강아지라는 존재의 속성과 동작을 코드로 표현하는 도구가 바로 클래스입니다.

코랩에서 새 코드 파일을 준비하고 이름은 '7일차\_클래스실습.ipynb'로 합니다. 빈 셀에서 다음 코드를 입력합니다.

```
CODE
class Dog: ①
    def __init__(self, name, age): ②
        self.name = name
        self.age = age
    def bark(self):
        return f"{self.name}가 멍멍 짭니다." ③
    def get_age(self):
        return self.age
```



- ① 예제 클래스 Dog는 `__init__` 라는 초기화 메서드와 `bark`, `get_age` 메서드로 구성되어 있습니다.
- ② `__init__` 메서드: 클래스에서 객체(인스턴스)가 만들어질 때 자동으로 호출되는 메서드로 보통 이 메서드에서 속성(Attribute)을 정의하고 초기값을 설정합니다. `self`는 클래스에서 자기 자신(인스턴스)을 가리키는 키워드입니다. 속성으로 `name`과 `age`를 저장합니다.
- ③ `bark`와 `get_age` 메서드: 메서드(Method)는 클래스에서 정의한 함수입니다. 이 메서드로 객체가 어떤 동작을 수행할지 정의합니다.

이제 정의한 클래스를 사용하겠습니다. 빈 셀에서 다음 코드를 작성합니다.

#### CODE

```
dog1 = Dog(name="초코", age=3) ①
dog2 = Dog(name="바둑이", age=5)
print(dog1.bark()) ②
print(dog2.get_age())
```

#### OUTPUT

```
초코가 멍멍 짭니다.
5
```

- ① 클래스 Dog를 이용해 dog1 인스턴스를 만듭니다. `name` 속성은 '초코', `age` 속성은 3입니다.
- ② 인스턴스 dog1의 메서드 `bark`를 실행해 결과를 출력합니다.

Dog 클래스에서 정의한 속성과 메서드가 객체(인스턴스)를 통해 어떻게 동작하는지 확인할 수 있습니다. 클래스는 구조를 정의하고 객체는 정의한 구조에 따라 동작하는 실제 주체임을 쉽게 이해할 수 있습니다.

### 클래스와 객체의 관계

클래스는 설계도이고 객체(인스턴스)는 그 설계도로 만든 실제 대상입니다. 건물을 짓기 위해 설계도가 필요하듯이 객체를 만들기 위해서는 클래스라는 설계가 필요합니다. 비유를 하자면 클래스는 빵틀이고 객체는 그 빵틀로 찍어낸 빵과 같다고 할 수 있습니다. 하나의 빵틀로 여러 개의 빵을 찍어낼 수 있듯이 하나의 클래스에서 여러 개의 객체를 만들 수 있습니다.

Dog 클래스는 '강아지'라는 객체의 공통 설계도입니다. 이 설계도에는 이름(name), 나이(age)와 같은 속성과 짖는다(bark), 나이를 알려 준다(get\_age)와 같은 동작이 있습니다. 설계도를 바탕으로 각각 독립적인 속성과 메서드를 사용하는 별개의 서로 다른 객체(인스턴스), 즉 예제처럼 dog1(초코), dog2(바둑이)를 생성할 수

있습니다. 공통의 설계도를 바탕으로 만들었지만, `dog1`과 `dog2`는 서로 다른 이름과 나이를 가진 개별적인 존재입니다.

클래스는 개념적인 정의이고 객체는 구체적인 실체입니다. 이 개념을 이해하면 앞으로 딥러닝 모델의 클래스를 설계할 때 하나의 설계도만으로 다양한 모델 객체를 만들어 사용할 수 있습니다.

## 모델을 클래스로 선언하기

딥러닝은 모델도 하나의 객체처럼 다룹니다. 예를 들어 강아지 클래스는 이름, 나이 같은 속성과 짖기, 먹기와 같은 동작을 합니다. 마찬가지로 딥러닝 모델도 입력층, 은닉층, 출력층으로 구성되어 학습하고 예측하는 기능을 수행합니다.

이번 절에서는 지금까지 배운 모델을 클래스로 선언하는 방법을 알아봅니다. 딥러닝에서는 회귀와 분류 모델을 각각 나누어 설명하지만, 클래스로 모델을 선언하는 방식에서는 차이가 없습니다. 이 절 마지막에서는 클래스로 선언한 모델로 간단한 실습을 진행하겠습니다.

### 기본 모델 설계

먼저 기본 모델을 클래스로 선언합니다. 모델 구조를 간단하게 직접 선언하는 종전의 방식과 클래스로 선언하는 방식을 비교하며 설명합니다.

#### 직접 선언 방식

`nn.Sequential` 함수로 모델을 선언하는 종전의 방식은 다음과 같습니다.

```
CODE
n_feature = 10
model = nn.Sequential(nn.Linear(n_feature, 3),
                      nn.ReLU(),
                      nn.Linear(3, 1)
                      ).to(device)
```

지금까지 모델링했던 방식입니다. `nn.Sequential` 함수는 복잡하지 않고 간단한 형태의 모델을 빠르게 정의할 때 사용하는 방법으로 각각의 층을 순차적으로 연결해 데이터를 처리합니다(입력 → 은닉층 → 활성화 함수 → 출력층의 순서). 그러나

nn.Sequential 함수를 이용한 모델링에서는 복잡한 모델 구조(예: 병렬 연결, 조건문 처리 등)를 정의하기 어렵고 조건부 실행이나 반복문을 활용한 모델 설계에도 제한이 있습니다.

직접 모델을 선언하는 방식은 주로 구조가 단순한 모델(예: 선형회귀, 간단한 분류 문제)을 설계하거나 다양한 실험에서 모델을 빠르게 테스트할 필요성이 있을 때에 한정해 사용합니다.

### 클래스 선언 방식

종전의 모델 코드를 클래스 선언 방식으로 바꾸면 다음과 같습니다.

#### CODE

```
class NeuralNet(nn.Module): ①
    def __init__(self, n_feature): ②
        super(NeuralNet, self).__init__() ③
        self.fc1 = nn.Linear(n_feature, 3) ④
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(3, 1)
    def forward(self, x): ⑤
        x = self.fc1(x) ⑥
        x = self.relu(x)
        x = self.fc2(x)
    return x
```

```
model = NeuralNet(n_feature).to(device) ⑦
```

- ① NeuralNet이라는 이름으로 새 클래스를 정의합니다. 이 클래스는 파이토치에서 제공하는 nn.Module을 상속으로 받습니다. nn.Module은 파이토치에서 신경망을 만들기 위한 기본 틀로서 모든 모델은 이 클래스를 기반으로 만들어집니다. 파이토치에서는 모델 클래스를 정의할 때 사용하는 기본 메서드가 두 개 있습니다. 즉, \_\_init\_\_와 forward 메서드입니다.
- ② \_\_init\_\_ 메서드(초기화 메서드): 클래스가 처음 만들어질 때(객체를 생성할 때) 자동으로 실행되는 함수입니다. 이 함수는 모델에 필요한 층(Layer)들을 정의합니다. 여기서 입력값 self는 클래스 자기 자신(인스턴스)을 가리키는 변수이고, n\_feature는 이 클래스로 모델을 만들 때 외부에서 받는 입력 특징(feature)의 개수입니다.
- ③ super(NeuralNet, self).\_\_init\_\_: 지금 만드는 NeuralNet은 nn.Module을 상속받아 만든 클래스입니다. 그런데 nn.Module에는 모델을 관리하고 학습할 때 꼭 필요한 기능들(예: 모델 파라미터 저장, GPU로 이동, 순전파(forward) 실행 등)이 이미 구현되어 있습니다. 따라서 super(NeuralNet, self).\_\_init\_\_를 실행해야 nn.Module에 있는 기본 기능을 NeuralNet 모델 클래스에 포함할 수 있습니다.
- ④ 첫 번째 층을 완전 연결(Fully Connected) 방식으로 정의합니다. 입력 데이터의 특징 개수는 n\_feature이고 출력 노드(뉴런) 수는 3으로 지정합니다. 예를 들어 입력 데이터가 [x1, x2,

x3, x4]로 4개의 특징이 있다면 이번 층에서 3개의 출력값으로 변환됩니다.

- ⑤ forward 메서드(순전파 메서드): 입력 데이터를 받아 각각의 층을 통과하는 과정(순전파)을 정의합니다. `__init__` 메서드에서 설정한 층과 활성화 함수 등 모델의 구성 요소를 준비하는 곳입니다. forward 메서드에서 준비된 요소를 이용해 모델 구조를 설계합니다. 모델 구조대로 입력 데이터는 단계별로 층을 통과하고 출력층에서 최종 출력값을 반환합니다. 당연한 이야기지만, 이 메서드는 데이터 처리의 단계를 지정하므로 순서가 중요합니다.
- ⑥ 입력 데이터를 첫 번째 완전 연결층에 통과시킵니다.
- ⑦ `NeuralNet(n_feature)` 클래스를 사용해 모델을 선언합니다. 이때 `n_feature`를 입력값으로 넣습니다. `n_feature`는 ②에서 정의했습니다.

좀 복잡해 보이지요? 하지만 클래스를 사용하는 것이 장점이 많기 때문에 실무에서는 모델을 개발할 때 대부분 이 방식을 사용합니다.

딥러닝 모델에서 클래스를 사용할 때는 주로 복잡한 모델을 설계하는 경우, 코드의 재사용과 확장이 필요한 경우, 그리고 조건문이나 반복문이 필요한 경우입니다. 참고로 실제 딥러닝 프로젝트에서는 모델 구조는 클래스로 정의하고 학습, 평가, 데이터 세트는 함수로 구성하는 경우가 많습니다. 클래스를 사용하면 코드의 구조가 명확해지고 유지보수 및 확장성이 뛰어나며 다양한 실험을 진행하기에 유리하기 때문입니다. 모델 구조만 클래스로 정의하면 나머지 코드(학습, 평가, 데이터 로딩)는 얼마든지 유연하게 작성할 수 있습니다.



#### self란?

`self`는 클래스에서 자기 자신(인스턴스)을 가리키는 변수입니다. 클래스에서는 정의된 변수와 메서드를 사용할 때 반드시 `self`를 붙여 접근합니다. 쉽게 말해 `self`는 클래스에서 만든 자기 자신을 의미합니다.

예를 들어 다음과 같이 층을 선언할 때 `self.fc1`은 `NeuralNet` 객체에 속하는 변수(`fc1`)라는 의미입니다.

#### CODE

```
self.fc1 = nn.Linear(n_feature, 3)
```

`self`를 붙이지 않으면 함수 실행 후에 사라지지만, `self`를 붙이면 객체가 존재하는 한 계속 유지됩니다. 클래스에서 변수를 정의할 때 `self`는 객체 자신을 가리키는 '예약어'로서 이를 붙여야 속성이 객체에 저장됩니다.



## 여러 가지 모델 클래스 설계 방법

이번에는 좀 더 복잡한 구조의 모델을 클래스로 선언하는 몇 가지 방법을 살펴봅니다.

### 직접 선언 방식

복잡한 구조의 모델은 어떻게 다른지 직접 선언 방식과 클래스 선언 방식을 서로 비교하며 설명합니다. 다음 코드는 은닉층과 드롭아웃이 여러 개 포함된 직접 선언 방식의 모델입니다.

```
CODE
n_feature = 28 * 28
n_class = 10
model = nn.Sequential(nn.Flatten(),
                      nn.Linear(n_feature, 256),
                      nn.ReLU(),
                      nn.Dropout(p=.3),
                      nn.Linear(256, 64),
                      nn.ReLU(),
                      nn.Dropout(p=.3),
                      nn.Linear(64, n_class)
                      ).to(device)
```

이 코드는 MNIST 데이터를 펼치고(Flatten) 두 개의 은닉층에 드롭아웃을 추가하는 종전의 모델링 방식입니다. 드롭아웃 비율은 모두 0.3으로 지정했습니다.

다음 과정에서는 이와 같은 직접 선언 방식의 복잡한 모델을 클래스로 어떻게 선언하는지 그 방법을 몇 가지로 나누어 소개합니다.

### 클래스 선언 방식 1: 기본

앞에서 직접 선언하는 방식의 모델 구조를 그대로 모델 클래스로 선언하면 다음과 같습니다.

```
CODE
class NeuralNet(nn.Module):
    def __init__(self, n_feature, n_class): ①
        super(NeuralNet, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(n_feature, 256)
        self.fc2 = nn.Linear(256, 64)
```

```
self.fc3 = nn.Linear(64, n_class)
self.relu = nn.ReLU()
self.dropout = nn.Dropout(p=0.3) ②
```

```
def forward(self, x): ③
    x = self.flatten(x)
    x = self.fc1(x)
    x = self.relu(x)
    x = self.dropout(x)
    x = self.fc2(x)
    x = self.relu(x)
    x = self.dropout(x)
    x = self.fc3(x)
    return x
```

```
model = NeuralNet(n_feature=28*28, n_class=10).to(device)
```

- ① n\_feature(입력 데이터의 특징 개수), n\_class(최종 분류할 범주의 개수, 즉 출력 노드 수)를 입력으로 받습니다.
- ② nn.ReLU와 nn.Dropout(p=0.3)은 반복적으로 사용하므로 한 번만 정의합니다.
- ③ x(입력 데이터)를 입력으로 받아 순차적으로 처리하면서 최종 결과를 반환합니다.

NeuralNet 클래스의 `__init__` 메서드는 모델에서 사용할 함수를 준비하고 외부에서 입력값을 받는다는 점, `forward` 메서드는 모델의 구조를 구성한다는 점을 꼭 기억하기 바랍니다. 특히 `forward` 메서드는 모델 구조에 따라 입력 데이터가 각각의 층과 활성화 함수, 드롭아웃 함수를 거치는 순차적인 과정을 정의합니다. 입력에서 출력까지 순차적으로 흘러가는 흐름을 순전파(Feed Forward)라고 합니다.



#### 드롭아웃 비율을 다르게 정의할 수 있나요?

드롭아웃 비율을 은닉층마다 다르게 지정하고 싶다면 해당 은닉층 드롭아웃 정의에서 개별적으로 정의하면 됩니다. 다음은 드롭아웃 비율을 개별적으로 선언하는 코드입니다.

#### CODE

```
self.dropout1 = nn.Dropout(p=0.2)
self.dropout2 = nn.Dropout(p=0.3)
```

은닉층마다 드롭아웃 비율을 얼마든지 다르게 지정할 수 있지만, 실무에서는 드롭아웃 비율은 동일하게 지정하는 것이 일반적입니다.

## 클래스 선언 방식 2: 중첩

이번에는 forward 메서드에서 함수를 중첩해 선언하는 방식입니다. 클래스 선언 방식 1의 forward 메서드를 다음과 같이 수정하면 됩니다.

CODE

```
def forward(self, x):
    # 입력 데이터의 순전파 흐름 정의
    x = self.flatten(x)
    x = self.dropout(self.relu(self.fc1(x))) ①
    x = self.dropout(self.relu(self.fc2(x)))
    x = self.fc3(x)
    return x
```

① self.dropout(self.relu(self.fc1(x))): self.fc1(x)의 처리 결과를 ReLU 함수를 거치게 한 다음, 드롭아웃 비율(self.dropout)을 적용합니다. 세 줄에 걸쳐 작성했던 코드를 단 한 줄로 간결하게 표현했습니다.

앞의 코드와 동작 결과는 동일하지만, 이 방식에는 몇 가지 장점이 있습니다.

우선 불필요한 중간 변수의 생성을 줄입니다. x1, x2 같은 중간 변수를 여러 개 만들지 않아도 되므로 메모리 사용을 조금이라도 줄일 수 있고 코드 흐름도 한눈에 보입니다. 또한 연산 흐름을 직관적으로 파악할 수 있습니다. 데이터가 'Linear → ReLU → Dropout' 순서로 흐른다는 것을 바로 확인할 수 있습니다. 특히 신경망 구조가 반복되는 경우에는 층의 조합 패턴을 훨씬 더 쉽게 확인할 수 있습니다.

## 클래스 선언 방식 3: 모듈화(Sequential로 묶기)

클래스 선언에서 nn.Linear+nn.ReLU+nn.Dropout을 하나의 세트로 묶어 선언하는 방식입니다. 다음과 같이 코드를 작성하면 됩니다.

CODE

```
class NeuralNet(nn.Module):
    def __init__(self, n_feature, n_class): # 모델에 사용할 층 정의
        super(NeuralNet, self).__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Sequential(nn.Linear(n_feature, 256), nn.ReLU(),
                                   nn.Dropout(p=.3)) ①
        self.fc2 = nn.Sequential(nn.Linear(256, 64), nn.ReLU(), nn.Dropout(p=.3))
        self.fc3 = nn.Linear(64, n_class)

    def forward(self, x): # 입력 데이터의 순전파 과정 정의
        x = self.flatten(x)
```

```
x = self.fc1(x) ②
x = self.fc2(x)
x = self.fc3(x)
return x
```

- ① nn.Sequential 함수에서 층을 하나의 단위(Linear→ReLU→Dropout)로 묶어 선언합니다.
- ② forward 메서드에서 fc1(x)을 한 번 호출하면 내부에서는 Linear→ReLU→Dropout순으로 실행하므로 추가로 코드를 작성하지 않아도 연산이 이루어집니다.

클래스 선언 방식 2가 forward 메서드에서 여러 과정을 중첩해 표현했다면 방식 3에서는 층을 선언할 때 Sequential 함수로 묶어 줍니다. Sequential 함수로 묶으면 forward 메서드를 만들 때 구조가 좀 더 간결해지므로 개발자들이 이 방법을 많이 선호합니다.

지금까지 직접 설계하는 방식과 클래스를 활용한 세 가지 선언 방식을 비교해 살펴 보았습니다. 클래스로 선언하는 까닭은 모델의 코드를 효율적으로 설계하기 위함입니다. 서로 간의 모델 성능에는 큰 차이가 없습니다.

다음 소절에서는 모델을 클래스로 선언하는 몇 가지 방식 중 하나를 선택해 모델을 선언하고 학습하고 예측하고 평가하겠습니다.

## 클래스로 모델링하기 실습

지금까지 실습했듯이 환경 준비부터 데이터 전처리, 모델링 순서로 코드를 작성하면서 모델은 클래스로 설계해 보겠습니다. 실습은 '7일차\_클래스실습.ipynb' 파일에서 이어 작성합니다.

### 환경 준비~데이터 전처리

딥러닝 모델링 절차를 다시 한번 짚어 보면 환경 준비에서는 라이브러리 로딩→디바이스 준비→사용자 정의 함수 생성순으로 코드를 작성하고 실행합니다. 데이터 전처리에서는 MNIST 데이터 로딩→데이터 로더 준비→검증 및 테스트 데이터의 준비로 진행합니다.

환경 준비는 그동안 여러 번 반복해 작성했기 때문에 별도의 코드를 제시하지 않습니다. 5일 차 185~187쪽을 참조해 작성하거나 '5일차\_다중분류모델실습.ipynb' 파일에서 해당 코드를 복사해 사용해도 좋습니다. 여기서는 데이터 전처리 과정에서 시작합니다.



다음과 같이 데이터 전처리 코드를 입력합니다.

CODE

```
# MNIST 데이터 다운로드
from torchvision import datasets
from torchvision.transforms import ToTensor
train_dataset = datasets.MNIST(root="data", train=True, download=True,
                               transform=ToTensor())
test_dataset = datasets.MNIST(root="data", train=False, download=True,
                              transform=ToTensor())

# 학습 데이터로 데이터 로더 구성
batch_size = 64
train_loader = DataLoader(train_dataset, batch_size=batch_size)

# 검증 및 테스트 데이터 준비
## 검증, 테스트 데이터 분할
x_val, x_test = test_dataset.data[:5000], test_dataset.data[5000:]
y_val, y_test = test_dataset.targets[:5000], test_dataset.targets[5000:]

## 스케일링
x_val = x_val / 255
x_test = x_test / 255

## 차원 맞추기
x_val = x_val.view(5000, 1, 28, 28)
x_test = x_test.view(5000, 1, 28, 28)
```

코드로 MNIST 데이터 불러오기, 데이터 로더 구성하기, 검증과 테스트 데이터 나누기, 스케일링과 차원 맞추기와 같은 전처리 작업을 통해 딥러닝 모델링에 필요한 데이터를 모두 준비했습니다.

### 클래스로 모델링하기

다음은 모델링을 진행합니다. 모델링은 모델 선언, 학습 및 학습 곡선, 예측 및 평가순으로 진행합니다.

먼저 모델 클래스를 준비합니다. 앞에서 설명한 모델 클래스 중 '클래스 선언 방식 1: 기본' 코드를 작성합니다.

CODE

```
class NeuralNet(nn.Module):
    def __init__(self, n_feature, n_class):
        super(NeuralNet, self).__init__()
```

```

self.flatten = nn.Flatten()
self.fc1 = nn.Linear(n_feature, 256)
self.fc2 = nn.Linear(256, 64)
self.fc3 = nn.Linear(64, n_class)
self.relu = nn.ReLU()
self.dropout = nn.Dropout(p=0.3)

def forward(self, x):
    x = self.flatten(x)
    x = self.fc1(x)
    x = self.relu(x)
    x = self.dropout(x)
    x = self.fc2(x)
    x = self.relu(x)
    x = self.dropout(x)
    x = self.fc3(x)
    return x

```

---

여전히 길고 복잡하게 느껴질 수 있습니다. 하지만 몇 번 반복해서 작성하다 보면 익숙해집니다. 이어서 모델 선언→손실 함수와 옵티마이저 준비→학습→학습 곡선→예측→평가순으로 코드를 작성합니다.

#### CODE

```

# 모델 선언
model = NeuralNet(n_feature=28*28, n_class=10).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.001)

# 학습
epochs = 10
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
    val_loss, _ = evaluate(x_val, y_val, model, loss_fn, device)
    tr_loss_list.append(tr_loss)
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")

# 학습 곡선
dl_learning_curve(tr_loss_list, val_loss_list)

# 예측 및 평가
_, pred = evaluate(x_test, y_test, model, loss_fn, device)
pred = nn.functional.softmax(pred, dim=1)
pred = np.argmax(pred.numpy(), axis = 1)
print(classification_report(y_test, pred))

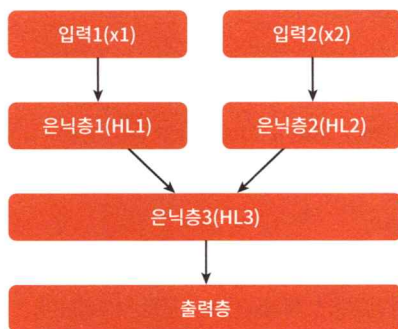
```

---

이번 예제는 모델을 클래스로 선언하는 것에 초점을 두고 있기 때문에 모델링 과정의 학습 및 학습 곡선, 그리고 예측 결과 등은 따로 설명하지 않겠습니다. 하지만 여러분은 직접 실행하고 그 결과도 확인하길 바랍니다.

## 다중 입력 구조의 모델 클래스 만들기

이번에는 보스턴 집값 데이터를 이용해 [그림 7-1]과 같이 특징(feature)을 두 개의 그룹에서 입력받는 모델을 구성합니다. 이 예시는 4일 차의 첫 번째 절 ‘은닉층에 대한 비즈니스 관점 이해’를 설명할 때 나온 모델 구조입니다. 은닉층에서 일어나는 일이 무엇인지 설명할 때 살펴본 내용인데, 기억이 나지 않는다면 4일 차 137쪽 [그림 4-1]을 참고하기 바랍니다.



[그림 7-1] 다중 입력 모델 구조

[그림 7-1]의 모델 구조를 보면 특징을 그룹화한 데이터 세트( $x_1$ ,  $x_2$ )를 구성하고 이 데이터 세트를 입력으로 받는 은닉층도 구성합니다(HL1, HL2). 계속해서 두 은닉층을 하나로 합쳐(Concatenate) 하나의 은닉층(HL3)으로 연결하고 마지막으로 출력층을 쌓습니다.

이제 [그림 7-1]의 구조로 모델을 구성하고 학습시키겠습니다. 환경 준비, 데이터 전처리, 모델 클래스 구성, 학습 및 평가 과정순으로 살펴봅니다. 실습은 ‘7일차\_클래스실습.ipynb’ 파일에서 이어 작성합니다. 환경 준비(라이브러리 로딩, 디바이스 준비, 함수 생성)는 이전과 동일하므로 여기서는 데이터 전처리부터 작성합니다.

### 데이터 전처리

보스턴 집값 데이터 세트를 불러와 집값에 영향을 주는 요인을 직접 요인과 간접

요인으로 나누고 각각의 데이터 세트를 구성합니다. 계속해서 모델 학습에 필요한 데이터 전처리 작업을 진행합니다.

### 데이터를 불러와 x1, x2로 나누기

보스턴 집값 데이터 세트를 불러옵니다. 그리고 목표값(target)을 기준으로 데이터를 x와 y로 분할한 다음, x를 직접 요인과 간접 요인 특징으로 각각 분할합니다.

#### CODE

```
# 1. 데이터 로딩
path = 'https://bit.ly/ds_boston_csv'
data = pd.read_csv(path)

# 2. 데이터 세트 분할: x, y
target = 'medv'
x = data.drop(target, axis = 1)
y = data[target]

# 3. x1, x2 나누기
x1_cols = ['rm', 'age', 'zn', 'tax', 'indus'] ①
x2_cols = ['crim', 'lstat', 'nox', 'dis', 'ptratio', 'rad'] ②
x1 = data.loc[:, x1_cols] ③
x2 = data.loc[:, x2_cols] ④
```

- ① 집값의 직접적인 요인에 해당하는 특징들을 리스트로 저장합니다.
- ② 집값의 간접 요인에 해당하는 특징들을 리스트로 저장합니다.
- ③ 직접 요인 특징은 x1에 저장합니다.
- ④ 간접 요인 특징은 x2에 저장합니다.

두 개의 입력 데이터(x1, x2)와 목표값 데이터(y)를 준비했습니다. 이어서 각각의 데이터를 스케일링합니다.

### 스케일링

x1과 x2는 각각 다른 특징들로 구성된 데이터 세트이므로 스케일러도 따로따로 만들어 적용해야 합니다. 다음 코드를 작성합니다.

#### CODE

```
from sklearn.preprocessing import StandardScaler
scaler1 = StandardScaler() ①
scaler2 = StandardScaler()
x1 = scaler1.fit_transform(x1)
```

```
x2 = scaler2.fit_transform(x2)
y = y.values
```

① 지금까지는 주로 MinMaxScaler를 사용했는데, 이번에는 StandardScaler를 이용합니다. 두 스케일러 중 어떤 게 더 낫다고 단정지을 수는 없습니다. 둘 다 테스트해 보고 성능이 더 나은 방법을 선택하면 됩니다. 다만 이미지 데이터는 0~255 사이의 픽셀 값이므로 주로 MinMaxScaler를 사용하고, 다른 종류의 데이터들은 MinMaxScaler나 StandardScaler 둘 다 많이 사용합니다.

입력 데이터 x1과 x2를 각각 스케일링했습니다. 사용자 정의 함수인 make\_DataSet 함수는 넘파이 배열을 입력으로 받는데, fit\_transform 메서드를 적용하면 넘파이 배열로 저장하므로 추가적인 변환이 필요 없습니다. 따라서 목푼값 데이터(y)를 별도의 넘파이 배열로 변환하는 코드(y.values)만 추가로 작성했습니다.

### train, test 분할

지금까지는 train\_test\_split 함수에 x, y만을 넣고 분할했습니다. 이번에는 x1, x2, y를 넣고 분할합니다.

#### CODE

```
x1_train, x1_test, x2_train, x2_test, y_train, y_test = train_test_split(x1, x2, y,
                                                                           test_size=0.2)
```

train\_test\_split 함수의 수행 결과는 총 6개(x1\_train, x1\_test, x2\_train, x2\_test, y\_train, y\_test)로 입력 순서(x1, x2, y)에 따라 데이터 세트가 train과 test로 나누어집니다. 즉, x1을 x1\_train과 x1\_test로, x2를 x2\_train과 x2\_test로, 마지막으로 y를 y\_train과 y\_test로 나눕니다.

### 데이터 로더 생성

x1, x2, y를 하나의 텐서 데이터 세트로 묶어 데이터 로더를 생성하려면 make\_DataSet 함수를 조금 수정해야 합니다. 다음과 같이 make\_DataSet 함수를 수정합니다.

#### CODE

```
def make_DataSet(x1_train, x1_test, x2_train, x2_test, y_train, y_test, batch_size = 32):

    # 데이터 텐서로 변환
    x1_train = torch.tensor(x1_train, dtype=torch.float32) ①
    x2_train = torch.tensor(x2_train, dtype=torch.float32) ②
```



```

y_train = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
x1_test = torch.tensor(x1_test, dtype=torch.float32)
x2_test = torch.tensor(x2_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).view(-1, 1)

# TensorDataset 생성: 텐서 데이터 세트로 합치기
train_dataset = TensorDataset(x1_train, x2_train, y_train) ③

# DataLoader 생성
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle = True)
return train_loader, x1_test, x2_test, y_test ④

# 학습 데이터의 데이터 로더, 테스트 텐서 데이터 준비
train_loader, x1_test, x2_test, y_test = make_DataSet(x1_train, x1_test, x2_train,
x2_test, y_train, y_test, batch_size = 32) ⑤

```

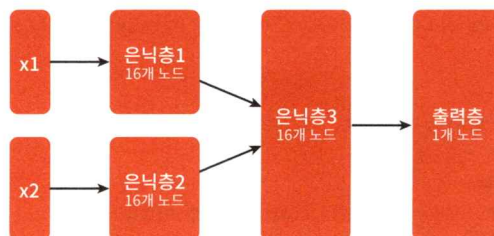
①② x1\_train과 x2\_train을 각각 텐서로 변환합니다.  
 ③ x1\_train, x2\_train, y\_train을 한꺼번에 텐서 데이터 세트로 묶어 줍니다.  
 ④ train\_loader와 x1\_test, x2\_test, y\_test 텐서를 반환합니다.  
 ⑤ 수정한 make\_DataSet 함수를 사용해 train\_loader와 테스트 텐서 데이터 세트(x1\_test, x2\_test, y\_test)를 준비합니다.

다중 입력에 맞게 데이터를 준비하는 함수 make\_DataSet를 수정하고 이 함수를 사용해 데이터 로더와 테스트 데이터를 만들었습니다. 이어서 모델 클래스를 사용해 모델링을 수행합니다.

## 모델링

### 모델 클래스 생성

이제 다중 입력 모델을 설계하겠습니다. 모델 구조는 [그림 7-2]와 같습니다. x1과 x2는 각각 은닉층1, 2로 입력됩니다. 은닉층1, 2의 16개 노드(합쳐서 32개 노드)의 결과는 은닉층3(16개 노드)을 통과하고 출력층에서 최종 결과를 만듭니다.



[그림 7-2] 다중 입력 모델 구조 예시

[그림 7-2]의 다중 입력 모델 구조를 MultiInputNet이라는 이름의 클래스로 생성합니다

CODE

```
class MultiInputNet(nn.Module):
    def __init__(self, input1_dim, input2_dim): ①
        super(MultiInputNet, self).__init__()
        self.fc1_branch = nn.Sequential(nn.Linear(input1_dim, 16), nn.ReLU()) ②
        self.fc2_branch = nn.Sequential(nn.Linear(input2_dim, 16), nn.ReLU()) ③
        self.fc_combined = nn.Sequential(nn.Linear(32, 16), nn.ReLU(), nn.Linear(16, 1)) ④
    def forward(self, x1, x2): ⑤
        out1 = self.fc1_branch(x1) ⑥
        out2 = self.fc2_branch(x2) ⑦
        combined = torch.cat((out1, out2), dim=1) ⑧
        output = self.fc_combined(combined) ⑨
        return output
```

- ① \_\_init\_\_ 메서드는 첫 번째 입력 데이터의 특징 개수(input1\_dim)와 두 번째 입력 데이터의 특징 개수(input2\_dim)를 입력으로 받습니다.
- ②③ 두 입력 데이터를 처리하는 완전 연결층(Linear)과 ReLU 활성화 함수입니다. 두 입력 데이터의 특징 수(입력 차원 input1\_dim과 input2\_dim)를 받아 각각 16개의 노드와 연결하도록 설계합니다. 여기서 노드 수는 조절할 수 있습니다.
- ④ 두 은닉층의 결과(16+16)를 합치면 총 32개의 노드가 됩니다. 이 32개 노드를 결합 은닉층에서 다시 16개 노드로 줄이고 마지막 출력층에서는 1개의 값만을 예측 결과로 반환합니다. [그림 7-2]를 참고합니다.
- ⑤ 모델이 두 개의 입력(x1, x2)을 동시에 받도록 합니다.
- ⑥⑦ 두 입력(x1, x2)은 첫 번째와 두 번째 은닉층(self.fc1\_branch, self.fc2\_branch)을 각각 통과하면서 연산을 수행합니다.
- ⑧ 각각의 은닉층에서 나온 결과(out1, out2)를 열별로(dim=1) 합칩니다(Concatenate).
- ⑨ 합쳐진 은닉층의 연산 결과를 출력층(④)에 통과시켜 최종 예측값(출력)을 생성합니다.

정리하면 다중 입력 모델 클래스에서는 입력1과 입력2를 각각 은닉층과 ReLU 함수로 처리한 다음, 이 두 결과를 합쳐 하나의 입력으로 만듭니다. 이렇게 합쳐진 32개의 노드를 다시 은닉층을 거쳐 출력층으로 연결해 최종 예측값을 만듭니다. 입력을 나누어 처리하고 그 결과를 합쳐 하나의 흐름으로 이어주는 과정에 주의하기 바랍니다.

## train 함수

다중 입력 모델 구조를 설계했으니 이어서 모델을 학습시킬 차례입니다. 준비된 데이터 로더와 모델 클래스를 이용해 학습시키려면 먼저 `train`과 `evaluate` 함수도 다중 입력을 받도록 수정해야 합니다.

### CODE

```
def train(dataloader, model, loss_fn, optimizer, device):
    size = len(dataloader.dataset)  # 전체 데이터 세트의 크기
    num_batches = len(dataloader)  # 배치 크기
    tr_loss = 0

    model.train()
    for batch, (x1, x2, y) in enumerate(dataloader):  ❶
        x1, x2, y = x1.to(device), x2.to(device), y.to(device)  ❷
        pred = model(x1, x2)  ❸
        loss = loss_fn(pred, y)
        tr_loss += loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
    tr_loss /= num_batches
    return tr_loss.item()
```

- ❶ 다중 입력 모델이므로 데이터 로더에서 `x1, x2, y` 3개의 텐서를 하나의 배치 단위로 받습니다. 참고로 `enumerate` 함수를 이용하면 인덱스(배치 번호)도 함께 반환하므로 디버깅할 때 유용합니다.
- ❷ 세 개의 텐서(`x1, x2, y`)를 `device`에 할당합니다.
- ❸ `x1, x2`를 모델에 입력하고 예측합니다. 이때 순전파(forward)가 실행되면서 예측값(pred)을 반환합니다.

입력 두 개를 처리하는 부분 외에 나머지는 기존의 `train` 함수와 동일합니다.

## evaluate 함수

평가 함수 `evaluate` 역시 `x1, x2, y`를 입력으로 받도록 수정합니다.

### CODE

```
def evaluate(x1, x2, y, model, loss_fn, device):  ❶
    model.eval()
    with torch.no_grad():
        x1, x2, y = x1.to(device), x2.to(device), y.to(device)  ❷
        pred = model(x1, x2)  ❸
```

```
eval_loss = loss_fn(pred, y).item()
return eval_loss, pred
```

- ① evaluate 함수의 입력으로 x1, x2, y를 받습니다.
- ② 세 개의 텐서(x1, x2, y)를 CPU로 지정합니다.
- ③ x1, x2를 입력으로 받고 예측값을 반환합니다

### 학습부터 예측 평가까지

함수를 모두 수정했다면 이제 모델 클래스로 모델 인스턴스를 선언하고 손실 함수와 옵티마이저를 지정해 학습합니다.

#### CODE

```
# 모델 선언
input1_dim, input2_dim = len(x1_cols), len(x2_cols) ①
model = MultiInputNet(input1_dim, input2_dim).to(device) ②

# 오차 함수, 옵티마이저 지정
loss_fn = nn.MSELoss()
optimizer = Adam(model.parameters(), lr=0.01)

# 학습
epochs = 50
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
    val_loss, _ = evaluate(x1_test, x2_test, y_test, model, loss_fn, device) ③
    tr_loss_list.append(tr_loss)
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

- ① 다중 입력 모델은 입력이 2개(x1, x2)이므로 각각의 특징(feature) 개수를 확인해 변수에 저장합니다. 여기서 x1\_cols와 x2\_cols는 데이터 세트를 준비할 때 만든 특징 목록입니다.
- ② 정의한 MultiInputNet 클래스의 모델 인스턴스를 생성합니다. 입력 데이터의 특징 수를 각각 지정해 모델이 올바른 크기의 데이터를 입력으로 받도록 합니다.
- ③ evaluate 함수에서 검증용 데이터 x1\_test 등을 입력하고 오차를 계산합니다.

다중 입력 모델의 학습 코드를 보면 train, evaluate 함수의 입력 부분만 다르고 나머지는 기존 모델과 동일하다는 것을 알 수 있습니다.

모델 학습이 끝나면 테스트 세트로 예측하고 평가를 진행합니다. 이 부분은 코드만 제시하고 따로 설명을 하지는 않겠습니다.

**CODE**

```

from sklearn.metrics import *
_, pred = evaluate(x1_test, x2_test, y_test, model, loss_fn, device)
mae = mean_absolute_error(y_test.numpy(), pred.numpy())
mape = mean_absolute_percentage_error(y_test.numpy(), pred.numpy())
print(f'MAE : {mae}')
print(f'MAPE : {mape}')

```

지금까지 여러 개의 입력 데이터를 사용하는 모델 구조를 살펴보았습니다. 데이터 전처리에서 모델 설계, 학습과 검증으로 이어지는 과정이 처음에는 다소 복잡하게 느껴질 수 있습니다. 그러나 딥러닝은 다양한 형태의 데이터를 동시에 다루는 방식이 점점 더 일반화되고 있기 때문에 다중 입력 모델을 스스로 설계할 수 있는 능력을 키우는 일은 꼭 필요합니다.

## 정리

이번 차시에서는 딥러닝 모델링에서 모델 구조를 클래스로 구현하는 이유와 그 필요성을 살펴보았습니다. 클래스를 사용하면 복잡한 모델 구조를 깔끔하게 정의하고 반복되는 코드를 줄이고 유지보수를 훨씬 간편하게 할 수 있습니다. 한편 하이퍼파라미터나 층 구성 등을 한곳에서 관리할 수 있어 코드의 일관성과 가독성을 높이는 데도 매우 유용합니다.

딥러닝 프로젝트에서 모델 구조는 클래스로 정의하고 학습과 평가는 별도의 함수나 코드 블록으로 분리하는 일이 일반적입니다. 이렇게 나누어 구성하면 코드가 더 명확해지고 재사용성과 확장성도 자연스럽게 보장됩니다.

지금까지 7개의 차시로 구성된 딥러닝 기초 파트를 모두 공부했습니다. 여기까지 잘 따라온 분 모두 수고 많았습니다. 지금까지 배운 내용 가운데 조금 어려운 부분이 있었다면 반드시 복습하고 다음 파트로 넘어가기를 권합니다.

이어지는 3개의 차시에서는 지금까지 배운 딥러닝 모델링의 기초 위에서 이미지 모델링, 자연어 모델링 그리고 비지도학습의 기본기를 설명합니다. 이 3개의 과정에서 다양한 형식의 데이터를 다루고 실제 문제 해결에 적용할 수 있는 딥러닝 모델의 설계 방법 등을 배울 수 있습니다. 점점 확장되는 딥러닝 세계를 차근차근 탐험하며 자신만의 모델을 만드는 재미를 느껴 보길 바랍니다.